PROJECT1: Design and Implementation of the Hybrid FTP Application

Course Name	Internetworking Protocol
Project Title	Design and Implementation of the Hybrid FTP
Project Type	Group Project (2-3 members/group)
Programming Language	C/C++, Java, Python, C#, or any widely-adopted systems language
Deliverables	Source Code + Technical Report + Live Oral Defense
Report Format	Structured Technical Documentation (see Section 4)

1. Project Description

This project challenges students to design and implement a **Hybrid FTP (File Transfer Protocol) system** that decouples the control plane from the data plane — mirroring the architectural philosophy of the real-world FTP standard (RFC 959). Students will build a fully functional client–server application with two independent communication channels:

1.1 Control Channel — TCP

The control channel leverages **TCP sockets** to transmit commands, responses, and session state. TCP's connection-oriented, reliable delivery guarantees sequential command execution, stable session tracking, and accurate status reporting throughout the client's lifecycle.

1.2 Data Channel — UDP

All actual file payload is transmitted over **UDP sockets**. Since UDP is inherently unreliable, students must research and engineer a custom application-layer reliability sub-protocol built directly on top of UDP — without any external libraries — to provide: zero packet loss, corruption detection, duplicate elimination, and correct packet ordering.

1.3 Evaluation Levels

Basic Level

- Authentication Mechanism: Basic user identification and access verification.
- Data Type & Transmission Mode: ASCII text file handling.
- File Operations: Upload and download of a single file.
- Operating Mode: Single, fixed data-channel connection mechanism.

Advanced Level

- Data Diversity: Binary file handling (images, video, archives) without corruption.
- Directory Navigation & Tree Support: Traverse, list, and manage nested folder structures.
- Flexible Operating Modes: Active / Passive mode switching or automation.
- Concurrency Control: Multi-threaded or multi-process server that fully isolates client sessions.

Excellent Level

- Custom Reliable UDP Layer (RDT): ACKs, sequence numbers, timeout/retransmit (Stop-and-Wait, Go-Back-N, or Selective Repeat).
- Congestion / Flow Control: Sliding Window or equivalent mechanism to prevent network flooding.
- Data Integrity Verification: End-to-end MD5 / SHA-256 hash comparison pre- and post-transfer.

2. Requirements

2.1 General Implementation Rules

- Language: C/C++, Java, Python, C# or equivalent.
- Only native, low-level socket APIs bundled with the language runtime are permitted.
- No pre-built FTP frameworks, third-party transfer libraries (e.g. KCP, QUIC, libcurl FTP wrappers) may be used.
- The custom reliable UDP layer must be implemented from scratch by the student team.
- A clear CLI or GUI must report network states, commands issued, and transfer progress.

2.2 Approved FTP Commands

The following table defines the complete list of FTP commands accepted by this project. Each command must be transmitted over the **TCP control channel**. The **Level** column indicates the evaluation tier at which the command is expected to be functional.

Command	Syntax	Description
USER	`USER <username>`	Send the client's username to initiate an authentication session.
PASS	`PASS <password>`	Send the client's password to complete authentication.
QUIT	`QUIT`	Gracefully terminate the control connection and end the session.
NOOP	`NOOP`	No-operation; used as a keep-alive ping to prevent session timeout.
PWD	`PWD`	Print the server's current working directory path.
CWD	`CWD <path>`	Change the server's current working directory to the specified path.
CDUP	`CDUP`	Change the server's working directory to its parent directory.
MKD	`MKD <dirname>`	Create a new directory on the server at the current path.
RMD	`RMD <dirname>`	Remove an empty directory from the server.
LIST	`LIST [path]`	Return a detailed listing (name, size, type, permissions) of files and directories in the current or specified path.
NLST	`NLST [path]`	Return a plain name-only listing of files in the current or specified path.
STAT	`STAT [path]`	Return server status or, if a path is given, file/directory metadata.
SIZE	`SIZE <filename>`	Return the exact byte size of the specified file on the server.
MDTM	`MDTM <filename>`	Return the last modification timestamp of the specified file (format: YYYYMMDDhhmmss).
TYPE	`TYPE {A | I}`	Set the data transfer type: A = ASCII (text), I = Image/Binary.
MODE	`MODE {S | B | C}`	Set the transfer mode: S = Stream, B = Block, C = Compressed.
PORT	`PORT <h1,h2,h3,h4,p1,p2>`	Active Mode: Client specifies its IP and port for the server to open the data connection back to.
PASV	`PASV`	Passive Mode: Server opens a random port and returns its IP + port for the client to connect to.
RETR	`RETR <filename>`	Retrieve (download) the specified file from the server to the client via the data channel.

Command	Syntax	Description
STOR	`STOR <filename>`	Store (upload) a file from the client to the server using the current filename.
STOU	`STOU`	Store a file with a guaranteed unique server-generated filename to prevent overwrites.
APPE	`APPE <filename>`	Append the uploaded data to an existing file on the server; create it if absent.
DELE	`DELE <filename>`	Delete the specified file from the server.
RNFR	`RNFR <oldname>`	Rename From: specify the file to be renamed (must be followed by RNTO).
RNTO	`RNTO <newname>`	Rename To: complete the rename operation initiated by RNFR.
HASH	`HASH <filename>`	Request a cryptographic hash (MD5 or SHA-256) of the specified file for post-transfer integrity verification.
ABOR	`ABOR`	Abort the current data transfer in progress; data channel is reset.
HELP	`HELP [command]`	Return help text for all supported commands, or detailed usage for a specific command.

2.3 Standard Server Reply Codes

The server must respond to every client command using standard three-digit FTP reply codes over the TCP control channel:

Code	Category	Common Examples
1xx	Positive Preliminary Reply	125 Data connection already open; 150 File status okay, opening data connection.
2xx	Positive Completion Reply	200 Command OK; 220 Service ready; 221 Goodbye; 226 Transfer complete; 230 Login successful; 250 Requested file action OK.
3xx	Positive Intermediate Reply	331 Username OK, need password; 350 Requested file action pending RNTO.
4xx	Transient Negative Reply	421 Service unavailable; 425 Can't open data connection; 426 Connection closed; transfer aborted; 450 File unavailable.
5xx	Permanent Negative Reply	500 Syntax error; 501 Syntax error in parameters; 502 Command not implemented; 530 Not logged in; 550 File unavailable.

2.4 Technical Report Requirements

The technical report must contain all of the following numbered sections:

1. Application Scenario & Protocol Interaction — Sequence diagram of the full TCP + UDP lifecycle.
2. Project-Wide Data Structures — TCP control packet format, UDP custom header fields (sequence no., ACK, checksum, flags, payload length), session management structures.
3. Functional Workflows (Flowcharts) — Server thread-dispatch logic, reliable UDP sender/receiver state machines, Active/Passive mode toggle.
4. Task Assignment Matrix — Module owner and collaborators for every engineering component.

5. Self-Assessment & Peer Evaluation — Individual written evaluations plus an agreed contribution percentage totalling 100%.
6. GenAI Usage & Code Refinement Log (Mandatory Appendix) — Exact prompts, raw AI output, and documented refinements showing critical analysis.
7. Application Demo Evidence — Screenshots / logs of upload, download, hash comparison, connected-client table, and concurrent session test.

3. Grading Rubric

Assessment is conducted via an **Oral Defense (Viva Voce)** combined with a live software demonstration (Demo) and technical report review. The rubric below distributes 100% of the grade across four criteria.

Criteria	Weight	Unsatisfactory (0)	Satisfactory (Pass)	Good / Very Good	Excellent
Code Quality & Application Demo	**40%**	Fails to compile or crashes. Cannot transfer files. Used banned FTP libraries.	Runs safely; transfers ASCII files in a single, fixed operating mode; single-threaded server.	Stable binary transfers; multi-threaded concurrent server; full directory tree operations.	Optimised reliable UDP (ACK + timeout recovery); functional congestion control or end-to-end hash verification.
Theoretical Understanding (Oral Viva)	**30%**	Cannot distinguish TCP vs UDP. Does not understand control/data channel split. Cannot explain own socket calls.	Accurately explains socket workflow and TCP handshake; high-level rationale for hybrid design.	Articulates Active/Passive mode nuances; explains concurrency model; details every byte in the custom UDP header.	Flawless mastery of RDT states (Stop-and-Wait, GBN, SR); defends bandwidth optimisation strategies with mathematical precision.
Live Coding & On-the-Spot Debugging	**20%**	Cannot locate requested code blocks. Fails to fix minor logical errors when prompted.	Locates modules quickly; adjusts trivial parameters with minor guidance.	Efficiently identifies injected edge-case bugs; modifies control flow live without breaking stability.	Rewrites code segments live to satisfy arbitrary network constraints or edge-case scenarios set by the examiner.
Technical Documentation & GenAI Provenance	**10%**	Plagiarised or superficial report. Missing diagrams, flowcharts, or structure definitions. GenAI log absent despite evident AI code.	All mandatory sections present; diagrams use non-standard notations; GenAI appendix merely copy-pastes prompts.	Professionally organised; diagrams accurately reflect the live codebase; GenAI log clearly distinguishes AI output from student adjustments.	Industry-grade documentation; packet headers charted to individual bit/byte fields; GenAI appendix shows deep critical auditing of AI output.

4. Critical Evaluation Directives

4.1 Individual Grading Differentiation

For groups of two or more students, **grades are not distributed equally**. Final individual scores are determined by: (a) the Task Assignment Matrix, (b) Peer Evaluations, and (c) targeted individual questions during the Oral Viva. A student who cannot explain the module assigned to them — or who demonstrates poor understanding of the shared system architecture — **will be marked down individually**, regardless of whether the group application functions perfectly.

4.2 Zero-Tolerance Policy for Unverifiable Code (GenAI Audit)

The use of generative AI tools (ChatGPT, Gemini, Claude, GitHub Copilot, etc.) for learning and assistance is explicitly permitted. However, students must maintain absolute ownership of their code. If a student imports AI-generated code but is unable to explain its step-by-step logic, runtime execution, or underlying data structures during the oral examination, they will automatically receive an **Unsatisfactory (0 points)** for both the Theoretical Understanding and Live Coding criteria.

4.3 GenAI Documentation Requirements

All GenAI interactions must be transparently documented in the mandatory appendix, covering:

- Prompts Used: the exact input queries submitted to the AI tool.
- Raw GenAI Output: unedited code snippets or explanations returned.
- Refinement & Problem Solving: a critical analysis identifying errors, limitations, or banned libraries in the AI output, and documenting the manual debugging, refactoring, and optimisation performed by the student.

4.4 Academic Integrity Policy

This project is subject to the university's full academic integrity policy. In addition to the above:

- Direct copy-paste of another group's code — with or without cosmetic renaming — is treated as plagiarism and results in a zero for the entire group.
- All code must be version-controlled (e.g., Git). Examiners may request commit histories to verify incremental authorship.
- Any third-party code used beyond approved libraries must be fully cited and explained; undisclosed use constitutes academic dishonesty.

4.5 Demo & Submission Checklist

Before the Oral Defense, every group must verify:

1. Source code compiles and runs without errors on a clean machine.
2. At least one successful upload and one download are demonstrable live.
3. Server log displays connected client IPs, executed commands, and active session table.
4. Technical report includes all seven mandatory sections (see Section 2.4).
5. GenAI appendix is complete and honest.
6. Contribution percentages have been agreed upon and declared by all group members.
7. All demo evidence (screenshots, hash logs) is embedded in the report.

— End of Project Specification —